

throttLL'eM: Predictive GPU Throttling for Energy Efficient LLM Inference Serving

Andreas Kosmas Kakolyris^{†,‡}, Dimosthenis Masouros[‡], Petros Vavaroutsos[‡], Sotirios Xydis[‡] and Dimitrios Soudris[‡]

[†]ETH Zürich, [‡]National Technical University of Athens

akakolyris@ethz.ch, {dmasouros, petrosvav, sxydis, dsoudris}@microlab.ntua.gr

Abstract—As Large Language Models (LLMs) gain traction, their reliance on power-hungry GPUs places ever-increasing energy demands, raising environmental and monetary concerns. Inference dominates LLM workloads, presenting a critical challenge for providers: minimizing energy costs under Service-Level Objectives (SLOs) that ensure optimal user experience. In this paper, we present *throttLL'eM*, a framework that reduces energy consumption while meeting SLOs through the use of instance and GPU frequency scaling. *throttLL'eM* features mechanisms that project future Key-Value (KV) cache usage and batch size. Leveraging a Machine-Learning (ML) model that receives these projections as inputs, *throttLL'eM* manages performance at the iteration level to satisfy SLOs with reduced frequencies and instance sizes. We show that the proposed ML model achieves R^2 scores greater than 0.97 and miss-predicts performance by less than 1 iteration per second on average. Experimental results on LLM inference traces show that *throttLL'eM* achieves up to 43.8% lower energy consumption and an energy efficiency improvement of at least $1.71\times$ under SLOs, when compared to NVIDIA's Triton server. *throttLL'eM* is publicly available at <https://github.com/WilliamBlaskowicz/throttLL-eM>.

I. INTRODUCTION

Large Language Models (LLMs) have surged in popularity over the last couple of years, driven by advancements in deep learning and highlighted by the remarkable success of ChatGPT from OpenAI [3]. This rapid adoption, combined with the reliance of LLMs on high-end GPUs with significant power and energy consumption has raised concerns about their environmental impact. [44], [53]. Recent studies indicate that inference dominates the computational workload, accounting for over 90% of the overall LLM compute cycles within data centers [45], [48]. While the environmental impact of data centers is primarily attributed to hardware manufacturing [22], their operational footprint is also significant; the energy required for a single GPT-4 response can reach $\approx 3\text{Wh}$ [7]. As userbases for LLM services grow into the millions [1], their energy footprint is projected to become substantial, potentially reaching 1050 TWh by 2026 [24], rivaling entire countries.

Minimizing the energy footprint of LLM inference serving systems is crucial not only for environmental reasons but also from the providers' perspective. Efficient utilization of resources allows data center providers to reduce their operating costs, while also facilitating compliance to power budget constraints, enforced by contracts with utility companies [73]. However, reducing energy consumption often conflicts with the need for high performance. LLM-based services (e.g.,

ChatGPT) are highly interactive, as users submit their queries (e.g., questions) and await for “on-the-fly” responses. This user expectation of low latency is often quantified through Service-Level-Objectives (SLO), e.g., maintaining the 99th percentile of response latency under a specific threshold [46].

Designing energy-efficient LLM serving systems that operate under SLOs is challenging due to their inherent unpredictability compared to traditional neural networks. We identify three major barriers that complicate this task:

- 1) **Autoregressive nature of LLMs:** Unlike traditional feed-forward neural networks (e.g., CNNs) with fixed execution steps, LLMs exhibit inherent stochasticity in their inference process. Response generation proceeds iteratively, processing one token (word or sub-word unit) at a time until an End-of-Sequence (EOS) token is generated or a predefined maximum token limit is reached. This dynamic token generation length makes it difficult to precisely pre-allocate computational resources for each inference request.
- 2) **Dynamic Batch Composition:** Techniques like *inflight batching* [71] leverage the autoregressive nature of LLMs to improve throughput by allowing requests to enter and exit a batch dynamically. However, as shown in Sec. III-A1, different batch sizes introduce significant performance variability, possibly increasing time-between-tokens and end-to-end latency by up to 45%, thus complicating energy optimization under SLOs.
- 3) **Variable Memory Footprint:** LLMs exhibit variable memory footprint due to their autoregressive nature. Within each iteration, memory is dynamically reserved for the growing sequence length and only deallocated once a request is completed. This takes the form of a variably sized Key-Value (KV) cache, which stores the information associated with each token. While techniques like *paged attention* [31] have mitigated memory fragmentation issues, managing the KV cache remains challenging. Notably, increased KV cache usage can lead to an up to 18.2% performance degradation in the inference process (cf. Sec. III-B).

Managing the energy consumption of inference serving systems through traditional power-capping [34] and power-oversubscription [30], [44] techniques falls short in the case of LLMs. These solutions often operate in a static manner, leading to high tail latencies under load [74], potentially conflicting with the specified SLOs set by providers [53].

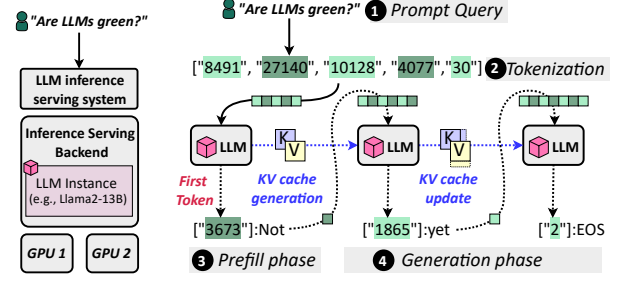
Moreover, techniques designed for energy optimization of traditional neural network inference serving systems [70], such as *delayed* [15] or *coordinated* [40] batching, cannot be directly applied to LLMs due to their unpredictability. Finally, race-to-idle strategies [29], [32] are also rendered inefficient since LLM inference serving systems typically experience a continuous stream of requests [46], [64], making it challenging to achieve true idle states for extended durations.

Given these limitations, dynamic GPU frequency scaling (throttling) forms a promising approach for optimizing energy efficiency in LLM inference serving. Unlike static power management techniques, frequency scaling offers a dynamic control knob, allowing the system to adjust its power consumption to the real-time workload demands of incoming inference requests. Crucially, frequency scaling can be applied at the granularity of individual LLM iterations (which typically span milliseconds), whereas traditional techniques operate on the scale of entire queries (which span several seconds). This fine-grained, iteration-level control allows throttling to be applied in a more granular manner, thus greatly enhancing energy efficiency while meeting predefined SLOs.

However, effectively applying frequency scaling to LLM inference remains challenging due to the inherent unpredictability in the text generation process exhibited by these models. While recent works address generation length prediction [23], [27], [51], [77], there is a notable gap in modeling the impact of LLM-intrinsic characteristics and leveraging these insights for energy-efficient LLM inference serving.

This paper presents *throttLL'eM*, an energy-efficient LLM inference serving framework. By leveraging dynamic GPU frequency scaling, *throttLL'eM* minimizes energy consumption while adhering to latency and throughput SLOs. The system relies on a projection mechanism that estimates KV cache utilization and batch size and a performance prediction model that forecasts system throughput at future LLM iterations. These predictions guide a throttling mechanism, which identifies the minimum frequency that meets target SLOs, thereby optimizing energy usage. *throttLL'eM* also features an autoscaling mechanism that adjusts the engine's parallelism level based on the incoming workload. Our contributions are:

- **Comprehensive analysis:** We conduct a thorough analysis of GPU frequency, KV cache usage, batch size, and instance parallelism, providing key insights into LLM performance and energy efficiency.
- **Predictive modeling:** We develop an analytical model that forecasts KV cache usage and batch size for future LLM iterations with *mean absolute prediction errors* of 2.26% and 0.19%, respectively. Additionally, we introduce a machine learning model that predicts iteration-level LLM performance with R^2 scores exceeding 0.97.
- ***throttLL'eM* framework:** We introduce *throttLL'eM*, a framework that leverages the aforementioned models for instance autoscaling and GPU frequency throttling, optimizing LLM inference energy consumption while meeting SLOs. *throttLL'eM* reduces energy consumption by an average of 24.7% without autoscaling, reaching 43.8% with autoscal-



(a) Inference serv-(b) LLM auto-regressivity. During *prefill*, the KV cache ing SW/HW stack. is generated. During *generation* it is updated.

Fig. 1: LLM **a)** serving stack and **b)** text generation process.

ing enabled, when compared to the state-of-the-art Triton inference server [43]. The proposed framework targets a p99th response time SLO equal to that of the baseline system running at peak load and a TBT SLO matching the human reading rate. *throttLL'eM* is publicly available at <https://github.com/WilliamBlaskowicz/throttLL'eM>.

II. BACKGROUND ON LLM INFERENCE

LLMs are neural networks with billions of parameters that heavily rely on the Transformer architecture [61]. Figure 1a shows an overview of a typical LLM inference serving stack. Inference servers (e.g., Triton [43], LMDeploy [14]) bridge the gap between users and pre-trained LLMs. They act as a frontend, receiving user queries and interacting with inference serving backends (e.g., TensorRT-LLM backend [42], vLLM [31]), which handle the deployment of LLM instances on the underlying infrastructure.

LLM text generation process: LLMs generate text in a step-by-step manner, one token (word or sub-word unit) at a time, as shown in Fig 1b. The process begins with a prompt from the user, which is tokenized before being fed into the model. The tokenization vector is passed to the LLM, generating the *first token*. This is known as the *prefill* or *prompt* phase and is compute-bound [46]. The LLM then enters an iterative loop until it generates an End-of-Sequence (EOS) token or reaches a maximum length limit (`max_tokens`). During each iteration, the LLM considers the prompt and previously generated tokens to calculate token occurrence probabilities and select a new one. The generated token is appended to the sequence, and the process repeats. This phase is referred to as *decoding* or *generation* and is known to be memory bound [4].

Key-Value (KV) cache: During the *prefill* phase the input tokens are processed, generating their initial representations (values) stored with their corresponding token identities (keys) in the KV cache. These key-value pairs are high-dimensional tensors, containing vectors for each attention head of the Transformer. For the decoder-only Transformer architecture used by many LLMs, these vectors remain identical across iterations and can thus be cached. In the generation phase, the LLM iteratively predicts tokens, checking the KV cache for

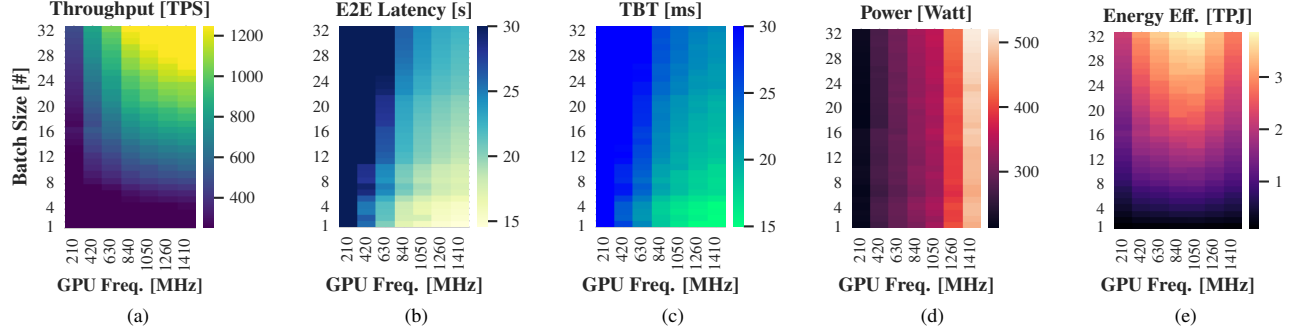


Fig. 2: Impact of different batch sizes and GPU frequencies on **a)** system throughput, **b)** end-to-end (E2E) latency, **c)** time between tokens (TBT), **d)** GPU power consumption and **e)** energy efficiency.

existing representations. If found, the cached representation is used; otherwise, a new representation is computed and stored.

Inflight Batching: *Inflight batching* [71] is an optimization technique which capitalizes on the autoregressive nature of LLMs, by allowing new requests to be added to or removed from an existing batch while it is being processed.

Instance Parallelism: Modern LLM serving frameworks [31], [42] support partitioning (sharding), enabling model distribution among many GPUs. This allows for executing large models on GPUs that would otherwise not be able to store LLM weights individually. Three key approaches exist to distribute the workload across multiple GPUs [41]. *Distributed Data parallelism (DDP)* creates identical copies of the model across multiple GPUs. *Tensor parallelism (TP)* splits the model’s weight tensors across multiple GPUs, enabling parallel computation for specific operations. Finally, *Pipeline parallelism (PP)* assigns consecutive LLM layers or segments to different GPUs, similar to instruction pipelining in CPUs.

LLM inference performance metrics: Several metrics have been proposed to evaluate LLM performance and serve as potential SLOs [46], [59]. *Time To First Token (TTFT)* measures the responsiveness of the inference engine, i.e., the time to generate the first token after query submission. Throughput metrics include *Time Between Tokens (TBT)*, *LLM iterations per second (IPS)*, and *tokens per second (TPS)*. *End To End latency (E2E)* represents the total time from query submission to completion. To quantify energy efficiency, we consider *tokens per Joule (TPJ)*, measuring the number of tokens generated per unit of energy expended.

III. ENERGY EFFICIENT LLM INFERENCE SERVING: CHALLENGES & OPPORTUNITIES

In this section we examine the impact of various factors that affect LLM performance and energy efficiency. We perform this analysis on a NVIDIA DGX A100 system, utilizing the Llama2-13B model as a representative example of LLMs¹, with a tensor parallelism of two, unless otherwise stated.

¹Since most LLMs are built upon a backbone of stacked Transformer blocks, similar conclusions can be drawn for other models.

A. Performance-Energy Trade-offs in LLM Inference

We investigate the interplay between performance and energy efficiency in LLM inference, aiming to identify the optimal operating points that align with specific performance metrics. Our analysis focuses on the following parameters: i) batch size, indicating the number of requests currently scheduled on the GPU; ii) GPU frequency, which directly impacts the latency of various LLM kernels; and iii) KV cache usage, representing the volume of data read by these kernels.

1) Impact of Batch Size and GPU Frequency: We first examine the impact of batch size and GPU frequency on the performance and power consumption of LLM inference. To this end, we spawn batches of different sizes, ranging from 1 to 32 requests wide. To avoid any variation or bias due to the context of the queries, all queries are identical with their prompt and generation lengths fixed to a specific value of 1 input token and 1024 generation tokens. Figures 2a-2e show the average throughput, E2E latency, TBT, power consumption, and energy efficiency during the lifetime of each batch.

Performance Analysis: Regarding performance related metrics (Fig.2a-2c), we first observe that throughput increases with batch size and GPU frequency, as shown in Figure 2a. Higher GPU frequencies enable faster processing of tokens, while larger batch sizes increase parallelism, contributing to higher overall throughput (TPS). Notably, when changing from a batch size of 1 at 210 MHz to a batch size of 32 at 1410 MHz, throughput experiences a threefold increase. This trend partially aligns with those observed for the E2E latency and TBT metrics. Figures 2b and 2c show that both E2E latency and TBT generally decrease with higher GPU frequencies due to reduced computation times. On the other hand, both metrics worsen as batch size increases, indicating that larger batch sizes incur overheads due to increased concurrent parallel processing and resource contention. Overall, both E2E and TBT approximately double when moving from the “high frequency, low batch size” to the “low frequency, high batch size” area.

Power Analysis: Figure 2d shows the power consumption

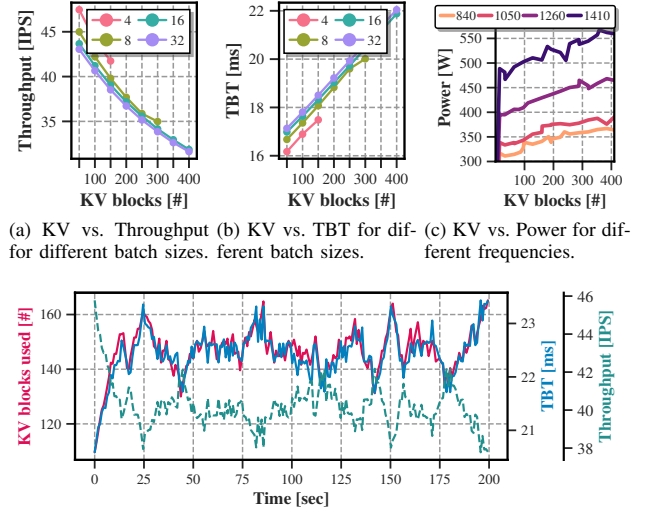
for different GPU frequencies and batch sizes. As expected, power increases with higher GPU frequencies due to increased switching activity and operating voltage required to sustain higher processing speeds. We note a greater than twofold increase in power draw between the lowest and highest available GPU frequencies. Interestingly, power consumption remains relatively constant across different batch sizes for a given GPU frequency, indicating that power draw is primarily influenced by the GPU's operating frequency rather than workload size. This uniformity of power consumption across batch sizes, combined with the variability in performance metrics, suggests that the system's energy efficiency can be optimized by carefully selecting the appropriate GPU frequency for the desired batch size, thereby balancing power draw and performance.

Energy Efficiency Analysis: To further support the above statement, Fig. 2e presents the energy efficiency of the system, i.e., the number of tokens generated per Joule. We see that processing larger batches improves energy efficiency for all frequencies. Counterintuitively to Fig. 2d, reductions in frequency do not always correspond to increased efficiency. We observe a clear sweet spot: efficiency increases with lower GPU frequencies up to a point (1050 MHz) before dropping again. Notably, using the maximum batch size with a lower, more efficient 1050 MHz frequency can yield a significant 37.4% boost in energy efficiency with minimal impact on performance (-6.25%, +8.26%, +5.41% throughput, E2E latency, and TBT respectively). For low frequencies (< 840 MHz) the reduction in power is offset by lower throughput, diminishing overall efficiency while simultaneously worsening performance. The above observations suggest that frequency scaling offers a valuable control knob for energy-efficient LLM serving. By dynamically adjusting the GPU frequency based on workload size (batch size) and aiming for the sweet spot where efficiency is maximized, we can achieve significant energy savings without sacrificing substantial performance. We note that optimal GPU frequency is architecture- and LLM-dependent and may vary with different generation lengths.

★ **Takeaways:** Batch size significantly impacts both performance and energy efficiency, with larger batches improving throughput and efficiency but worsening E2E latency and TBT. Low GPU frequencies are neither performant nor energy-efficient. The frequency of optimal energy efficiency differs from that of peak performance, highlighting frequency scaling as a promising mechanism for exploring this trade-off.

B. Implications of KV cache usage on LLM inference

Impact of KV on performance: Figures 3a and 3b show the impact of KV cache utilization on throughput (IPS) and TBT for different batch sizes as the token generation process progresses and the number of KV blocks allocated in the GPU's DRAM increases. The results reveal that both the number of allocated KV blocks and the batch size affect performance. We observe that as the number of blocks increases there is clear degradation in Throughput and a linear increase in TBT. This trend is consistent across all batch sizes. While this trend is similar across all batch sizes, smaller batches achieve better



(a) KV vs. Throughput (b) KV vs. TBT for different batch sizes. (c) KV vs. Power for different frequencies. (d) Timeline of KV cache utilization, TBT and Throughput. Batch size is kept equal to 32 and GPU frequency is set to maximum.

Fig. 3: Analysis on the impact and correlation of KV cache utilization to different performance metrics.

performance for the same number of allocated KV blocks, due to the fewer tokens that need to be generated at each iteration, resulting in reduced resource contention within the GPU.

Impact of KV on power consumption: Figure 3c shows the power consumption for a batch of 32 queries as the KV block count increases (reflecting the generation process) under different frequency levels. A positive correlation between KV blocks and power consumption is observed across all frequency levels, which is attributed to the increased number of DRAM reads per iteration. Notably, the power consumption curve steepens at higher frequencies as the number of KV blocks grows, indicating a more rapid power increase per additional KV block. This variation in gradient suggests that careful consideration of KV cache management and frequency scaling can lead to optimized energy efficiency.

KV/Performance correlation analysis: Our analysis has demonstrated a clear relationship between KV cache utilization, throughput, and TBT. To further examine this correlation, we conduct an inference experiment for 200 seconds while maintaining constant batch size (32) by introducing new requests of random generation lengths upon completion of existing ones, all under maximum frequency. Figure 3d shows that KV usage and TBT curves overlap, indicating a direct relationship, while throughput appears to be a mirrored image of KV cache usage. Statistical analysis yields a Pearson correlation of 0.92 between KV and TBT and -0.92 between KV and Throughput, indicating that KV usage can serve as an accurate proxy for modelling performance. Repeating the experiment at lower batch sizes increases the Pearson correlation, due to less interference from newly arrived requests entering the prefill phase and stalling the token generation process.

★ **Takeaways:** KV cache utilization significantly impacts both

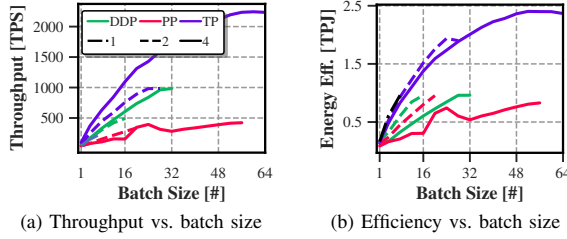


Fig. 4: Impact of different LLM partitioning approaches on a) throughput and b) energy efficiency.

performance and power consumption. The high correlation between KV utilization and performance-related SLOs emphasizes its effectiveness as a proxy for performance modeling.

C. LLM partitioning implications on performance and energy

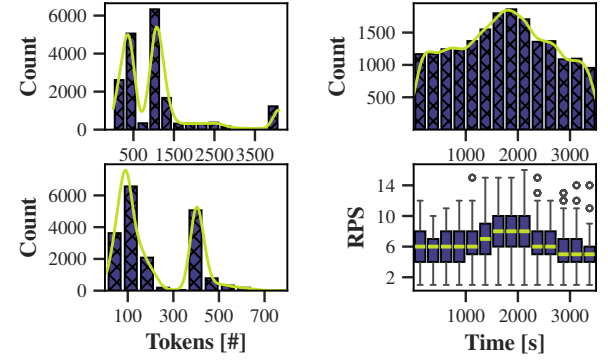
As described in Sec. II, LLMs support various techniques for partitioning the model across multiple GPUs, each impacting system efficiency differently. Figures 4a and 4b show the throughput (TPS) and energy efficiency (TPJ) respectively, for different partitioning techniques (DDP, PP, TP) and parallelism levels across various batch sizes on a single node. Regarding throughput, TP consistently outperforms DDP/PP, exhibiting a throughput improvement of $1.54/2.74\times$ and $1.79/6.26\times$ for parallelism levels 2 and 4 respectively, at the maximum batch size supported by all configurations. Notably, TP also supports larger attainable batch sizes, demonstrating its ability to handle more concurrent requests and provide increased serving capacity for an equal number of GPUs. As expected, increasing the level of parallelism enhances performance due to the increased computational resources and memory capacity provided by additional GPUs, which results in more available KV cache blocks, allowing the system to process a greater number of requests simultaneously. Efficiency-wise, TP again outperforms DDP and PP. However, smaller engine sizes provide better energy efficiency up to the maximum batch size they can support. Specifically, we see that TP2 achieves up to 9.66% higher TPJ compared to TP4, when running close to its maximum batch size, indicating that smaller engines should be prioritized whenever possible to optimize energy efficiency.

★ **Takeaways:** For intra-node scaling, TP configuration outperforms DDP and PP in both performance and energy efficiency. The trade-off between performance and energy efficiency at different levels of parallelism underscores the need for dynamic engine sizing based on real-time workload characteristics.

D. Unpredictability in LLM inference serving systems

Inference serving systems exhibit significant unpredictability due to the diversity of input prompts and workload traffic variations [46], [64]. To illustrate this, we analyze a 60-minute production trace from Azure LLM inference services [46].

Length Distribution: Figure 5a shows the distribution of prompt tokens (top) and generated tokens (bottom). Query



(a) Distribution of prompt (top) and generated tokens (bottom). (b) Histogram of requests (top) and min and max RPS per bin (bottom).

Fig. 5: Analysis of Azure's LLM inference trace [46].

prompts can reach up to 4000 tokens, with most queries falling between 0-1500 tokens. Similarly, generated tokens range from 10 to 700, with the majority between 100 and 400 tokens. The long-tail distributions observed for both prompt and generated tokens underscore the necessity for dynamic resource allocation strategies to handle the varying computational demands of different queries. Moreover, the unpredictable nature of token generation lengths further complicates resource management due to its direct impact on KV cache utilization and overall system performance (as shown in Sec. III-B).

Arrival Pattern: Figure 5b shows the distribution of requests (top) and RPS (bottom) over the 60-minute trace, segmented into 4-minute bins. The distribution exhibits non-uniformity, with a peak around the midpoint of the trace, indicating periods of lower and higher activity. Additionally, the request arrival rate shows high variability, with median values ranging between 5 and 8 RPS and the distribution extending up to 16 RPS. The minimum RPS per bin is one, indicating a continuous workload. Given that the generation phase typically spans multiple seconds, the system experiences sustained activity without idle periods, rendering race-to-idle techniques inefficient for energy optimization.

★ **Takeaways:** The long tails of token length distributions and high variability in arrival patterns necessitate dynamic resource allocation for optimal management. LLM inference systems rarely reach idle states, rendering race-to-idle techniques incapable of providing optimal energy efficiency.

IV. *throttLL'eM*: SLO-AWARE & ENERGY-EFFICIENT LLM INFERENCE SERVING

Building on insights from Sec. III, we introduce *throttLL'eM*, an energy-efficient, SLO-aware LLM inference serving system. *throttLL'eM* prioritizes critical SLOs, namely E2E and TBT, delivering energy-efficient inference by dynamically adjusting GPU frequency to match workload demands. Figure 6 shows an overview of *throttLL'eM*. New queries are processed by the *generation length prediction* component, which estimates the number of generated tokens. This prediction

enables accurate *KV cache usage and batch size projections*, allowing the system to evaluate potential SLO violations. If no violations are anticipated, the query is scheduled; otherwise, it is queued. Finally, *throttLL'eM* also includes an *autoscaling mechanism* to dynamically adjust LLM instance parallelism based on workload, and a *throttling controller* to optimize GPU frequency and energy consumption. Table I summarizes the terminology used in this section.

A. Generation Length Prediction

Accurate generation length prediction is essential for optimizing LLM inference, as it can mitigate KV cache overheads (cf. Sec. III-B) and prevent out-of-memory errors [27] due to dynamic memory allocations [31]. Given the direct correlation between the number of generated tokens and allocated KV blocks, length prediction is crucial for preventing such issues. However, accurately predicting generation length is challenging due to the stochasticity of LLMs, the diversity of input queries, and other factors influencing output length.

Recent works have proposed various approaches to tackle the aforementioned problem [23], [27], [51], [77]. In [23], [27], the authors model length prediction as a classification task, fine-tuning DistilBERT [54] and OPT-125M [75] respectively, to classify output length into “buckets”. In contrast, the authors of [51] use a regression model, appending a two-layer fully connected network to a BERT base model [18]. In [77], the authors propose the “*Perception in Advance*” method, where the model estimates the length of its response before generating the output. All approaches report errors up to $\approx 30\%$. In this paper, we consider a prediction model that estimates the expected generation length ($\widehat{r}_a$) of each input query. This predicted token generation length ($\widehat{r}_a$) is propagated to the KV cache usage & Batch Size projection component.

B. KV usage & Batch Size projection

The *KV usage & Batch Size projection* component generates vectors containing the projected KV cache utilization (KV) and batch size (B) at each future iteration until all currently scheduled requests terminate. By assuming an oracle-based generation length predictor and considering the iterative nature of the token generation process, it is possible to create accurate

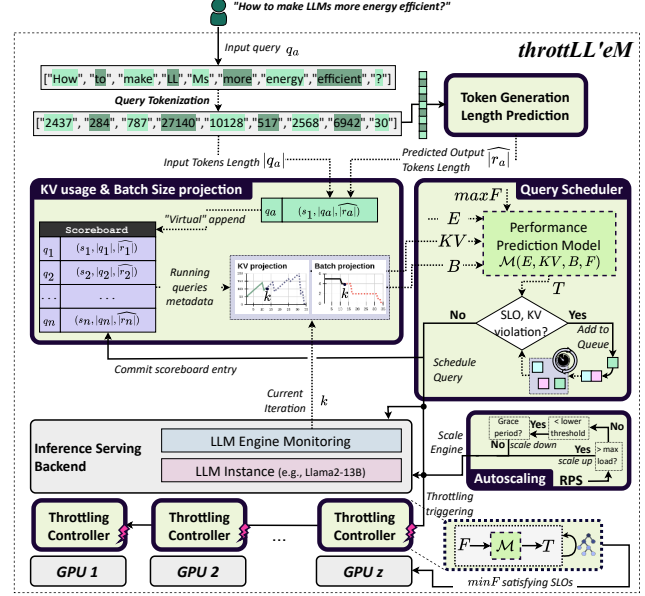


Fig. 6: *throttLL'eM* Overview

projections for these values over time. Specifically, since each request in the batch generates one token per iteration (each corresponding to a fixed KV cache block allocation size), we can predict the cumulative KV cache demands and batch sizes for upcoming iterations using an analytical model.

To keep track of scheduled requests, a *Scoreboard* is kept, containing the following metadata for each query (q_i) in the batch: i) the iteration at which the query was scheduled (s_i), ii) the input length of the query ($|q_i|$) and iii) its predicted generation length ($\widehat{r}_i$). Based on these values, we can determine that a query will complete its generation process at iteration $s_i + \widehat{r}_i$, i.e., $\widehat{r}_i$ iterations after it was initially scheduled. By repeating this process for each entry in the *Scoreboard*, we can identify the final iteration of all scheduled queries, allowing us to generate a vector $B = [B_1, B_2, \dots, B_j, \dots, B_n]$, where B_j is the projected batch size at each future iteration $j \in [1, n]$ and n is the iteration at which the last query will be completed.

A similar approach can be followed to calculate the *KV cache* utilization for each iteration. At iteration j , request q_i will have generated $j - s_i$ tokens². The KV cache also stores the precomputed values for the $|q_i|$ tokens that constitute the input of q_i . As each block has a capacity of N tokens, where N is a compile time parameter, the KV cache blocks allocated to request q_i ($KV_{q_i}[j]$) are given by Equation 1.

$$KV_{q_i}[j] = \begin{cases} \lceil (j - s_i + |q_i|)/N \rceil & , s_i \leq j < (s_i + \widehat{r}_i) \\ 0 & , \text{otherwise.} \end{cases} \quad (1)$$

The total KV cache block usage of the inference engine at each future iteration j can finally be calculated by summing

²Under the constraint $s_i \leq j < s_i + \widehat{r}_i$ which ensures that q_i is scheduled.

TABLE I: Terminology used in Section IV

Term	Definition
s_i	The iteration at which query q_i is scheduled
$ q_i $	The input length of query q_i
$\widehat{r}_i$	The predicted generation length of query q_i
KV	Vector of predicted KV cache usage for the LLM engine
KV_{q_i}	Vector of predicted KV cache usage for query q_i
B	Vector of predicted Batch size per iteration
T	Vector of predicted token generation throughput (IPS)
N	Number of tokens per KV cache block
$\widehat{t}_R(q_i)$	The estimated remaining time for query q_i
$t_{dead}(q_i)$	The deadline for the E2E SLO of query q_i
$\mathcal{M}$	The performance prediction model used by <i>throttLL'eM</i>
$\max F$	The maximum frequency supported by the GPU
E	The size of the engine (number of GPUs)

all KV_{q_i} . This is the KV of Figure 6, given by Equation 2, where $KV = [KV_1, KV_2, \dots, KV_j, \dots, KV_n]$.

$$KV[j] = \sum_{\forall q_i} KV_{q_i}[j] \quad (2)$$

Last, when a query terminates, its entry is struck from the *Scoreboard*, signalling the deallocation of all associated KV cache blocks. The introduction of this component allows us to project the usage of the KV cache and the batch size into the future, under the assumption that no new requests will arrive. We measure an average latency of < 2 ms for this component. **Projections for new queries:** When a new query q_a is submitted and before it is scheduled, its metadata are “virtually” appended to the *Scoreboard* with s_a being set to the current iteration k , thus, allowing us to assess how future KV cache usage and batch size would be affected by the immediate scheduling of the request. If the query is indeed scheduled by *throttLL'eM*'s scheduler (Sec.IV-C), this “virtual” change is committed to the *Scoreboard*, otherwise, it is rolled back.

C. LLM Query Scheduler

The *Query Scheduler* is responsible for orchestrating the execution of incoming requests on the LLM engine. To identify potential SLO violations, a *Performance Prediction Model* is employed. If no SLO violations are predicted, the query is scheduled for execution; otherwise, it is placed in a waiting queue until it can be served without compromising any SLO.

1) *Performance Prediction Model:* The *Performance Prediction Model* ($\mathcal{M}$) predicts the throughput of the system in terms of iterations per second (IPS). Building on the high correlation of throughput to KV cache usage, GPU frequency, and batch size, as demonstrated in Sec. III-A1 and Sec. III-B, the model leverages these metrics as input features to accurately forecast system performance at the iteration level.

Training data collection: Capturing the entire design space for training the predictive model is prohibitive due to i) the fine granularity of GPU frequencies (15 MHz steps); ii) the extensive range of possible values for KV block usage; and iii) the large batch sizes, which can reach up to 512 instances in few-parameter models with a high degree of parallelism, or on GPU infrastructures with higher memory capacity [4]. To address this challenge, we employ a systematic sampling approach to acquire a representative subset of the design space. Specifically, we employ a request generator that profiles the LLM inference engine across different batch sizes by spawning queries with predefined generation lengths, ensuring maximum utilization of the KV cache at the final iteration. For each batch size, the generator spawns a number of requests equal to the batch size to cover the entire KV cache usage range, ensuring that the edges of the profiling space are present in the dataset. A monitoring agent is deployed to continuously log KV cache usage, GPU frequency, batch size, and the experienced IPS every second. During this process, the GPU frequency is randomly changed (for high frequency range coverage) after each measurement is completed and before starting a new one, in order to ensure that the frequency remains consistent

for the duration of each measurement. This data collection process is repeated for all supported tensor parallelism (TP) levels, generating a dataset for each LLM that captures the relationship between the input variables and IPS.

Model selection & training: *throttLL'eM* adopts a machine learning (ML) approach to model the relationship between system metrics and IPS. Given that all input features are upper and lower bounded for a given LLM, we employ a Gradient Boosted Decision Tree [11], [21] as our ML model, which has been shown to provide accurate interpolation predictions for bounded datasets with multiple interacting features [36]. The model is lightweight (≈ 3 ms inference latency on CPU), meeting the design constraint of low inference latency. This is vital not only for enabling iteration-level predictions but also because the model resides in the critical path of the *Scheduler*. We use the following features as input to the model: i) engine size; ii) batch size; iii) KV cache usage; and iv) GPU frequency, training it to predict the throughput (IPS).

2) *Query Scheduling & Admission Control:* Upon receiving a new query, the *Scheduler* initiates an admission control process, using the projected batch (B) and KV cache size (KV) vectors, generated by the *KV usage & Batch Size projection* mechanism using the “virtual” *Scoreboard*. The *Scheduler* must determine if the new query can be scheduled without causing any SLO violations. This involves three checks:

1. KV cache assessment: The projected KV cache utilization (KV) is compared to the available KV cache capacity of the inference engine. If any value in KV exceeds this capacity, the query is queued to prevent significant performance degradation from KV block swapping to main system memory.

2. TBT SLO compliance: For queries meeting KV cache constraints, the *Scheduler* uses $\mathcal{M}$ to estimate throughput for each projected batch size ($B_j \in B$) and KV cache usage ($KV_j \in KV$) pair. The model is queried with: i) the current size of the LLM inference engine, ii) projected batch sizes B , iii) projected KV cache utilization KV , and iv) the maximum available GPU frequency, reflecting peak theoretical performance. The model predicts a vector T of maximum attainable throughput (IPS) per future iteration. Since TBT = 1/IPS, the *Scheduler* calculates the inverse values of T to form a vector T' , representing TBT per iteration. If the average value of T' exceeds the TBT SLO, the request is queued.

3. E2E SLO compliance: Given the TBT vector T' , we can determine the estimated time to reach a future iteration j , by calculating the vector $\widehat{T}_R = [T_{R_1}, T_{R_2}, \dots, T_{R_j}, \dots, T_{R_n}]$, i.e., performing a cumulative sum on the values of T' as follows:

$$\widehat{T}_{R_j} = \sum_{i=k}^{i \leq j} T'_i = \sum_{i=k}^{i \leq j} T_i^{-1} \quad (3)$$

For request q_i , finishing at iteration $l = s_i + \lceil r_i \rceil - k$, the remaining time is given from element T_{R_l} . The current iteration k is subtracted as future iterations are relative to k . Thus, for each q_i which is predicted to complete at iteration

l , the *current time* (t_{cur}) is added to T_{R_i} and compared to the *deadline* imposed by the E2E SLO ($t_{dead}(q_i)$):

$$\forall q_i, T_{R_i} + t_{cur} < t_{dead}(q_i) \quad (4)$$

For queued requests, this admission control process is repeated until the *Scheduler* decides that they can be safely admitted to the batch without causing violations. If all checks are successful, the newly arrived request is scheduled and any changes to the *Scoreboard* are committed. Otherwise, the request is queued and changes are rolled back. If a new request cannot be scheduled without violation of its own E2E SLO, but does not cause violations, it is scheduled but marked as “lost”, signaling the *Scheduler* to ignore it during future SLO validations.

D. LLM autoscaling mechanism

Effective autoscaling is crucial for optimizing both performance and energy efficiency in LLM inference systems. As shown in Sec. III-C, tensor parallelism (TP) forms the most efficient approach for scaling the LLM engine to multiple GPUs on the same node. To this end, *throttLL'eM*'s autoscaling mechanism relies on managing the TP of the LLM engine to match the incoming request rate (RPS), ensuring that the system operates efficiently under varying loads.

Autoscaling is driven by a 10-second interval monitoring agent that determines the minimum engine size required to handle the current workload. This decision is based on pre-characterized performance profiles, which define the maximum load each engine can sustain without introducing long tail latencies (cf. Sec. V-A). To mitigate the significant provisioning latency (>20s) of new inference servers, we employ “*shadow instancing*” [13], [19], thus masking switching overheads. This involves a “warm-up” phase, where new requests are served by the current engine until the new one becomes operational. Afterwards, a “transition” phase occurs, during which the old engine gracefully shuts down, while the new engine takes over, serving all new requests. During shadow instancing, both engines run simultaneously, consuming energy without providing peak performance capacity until fully loaded.

To regulate autoscaling and minimize energy waste, we establish a policy where newly spawned engines are given a “grace period” equal to their spawn time. During this period, the autoscaler may switch to a larger engine to adapt to sudden load spikes but may not switch to a smaller engine, preventing premature downscaling and unnecessary shadow instancing. The grace period is renewed each time the measured RPS is within the engine’s workload constraints. This policy prioritizes adherence to SLOs by always allowing scale-up decisions while minimizing scale-downs and unnecessary shadow instancing, thereby reducing energy consumption.

E. GPU Frequency Throttling Controller

The throttling controller is triggered upon admission of a new query to the inference engine. Its goal is to identify the minimum GPU frequency that adheres to the target SLOs, unlike the *Scheduler*, which validates SLO compliance at

maximum frequency. Since this component is invoked subsequent to successful SLO validation at maximum frequency, the existence of at least one SLO-compliant GPU frequency is guaranteed. The controller receives the *KV* and *B* vectors from the *KV and Batch projection* mechanism, performing a binary search over the GPU frequency range. This involves: 1) Recalculating T , T' , and $\hat{T}_R$ for each frequency; and 2) Checking TBT and E2E SLO compliance using the same procedure as the *Scheduler*. This process determines the minimum SLO-satisfying frequency. The selected frequency is applied to all the GPUs serving the LLM instance, thus, minimizing operational complexity and mitigating barrier synchronization issues due to stragglers [69]. If a “lost” request is present in the current batch, the binary search is bypassed, and the maximum GPU frequency is selected in a best-effort attempt to meet the “lost” SLO. We measure the combined latency of the *Scheduler* and *GPU frequency throttling* components at 35 ms under heavy load.

F. Discussion

Addressing generation length prediction errors: The effectiveness of *throttLL'eM* relies on accurate generation length predictions, yet such errors are unavoidable. Queries terminating earlier than expected primarily impact energy savings, as their SLOs could have been met at lower frequencies. Conversely, queries exceeding the predicted length risk violating SLOs. To mitigate this, we conservatively adjust the predicted generation length ($\lceil \hat{r}_i \rceil$) by a factor proportional to the error induced by the predictor. If the actual length surpasses the adjusted $\lceil \hat{r}_i \rceil$ during the generation process, the *Scoreboard* entry for the request is updated to the maximum length limit (`max_tokens`) supported by the model.

***throttLL'eM* and TTFT optimization:** *throttLL'eM* does not explicitly optimize the *prefill* phase, a compute-intensive part of LLM inference. While the *prefill* phase typically constitutes a smaller proportion of overall latency compared to the generation phase [46], it still represents a potential performance bottleneck. Increasing GPU frequency for a single iteration (upon scheduling a new query) would be an inefficient way to manage TTFT due to the frequency switching overhead (avg. 200 ms) compared to the short *prefill* phase duration (avg. ≈ 175 ms). The Splitwise framework [46] offers a promising alternative by decoupling the *prefill* and *generation* stages onto separate instances, allowing for independent optimization.

***throttLL'eM* scalability:** The proposed solution uses TP to distribute LLMs across GPUs, an approach shared with other concurrent works [44], [46]. TP offers low latency but requires high performance interconnects and may not scale beyond a single node. As discussed in [4], for larger models which need to be distributed across nodes, PP is often used for inter-node communication, while TP is employed intra-node to take advantage of its increased performance. In this case, the modelling methodology presented still applies within each node but additional components would be required in order to orchestrate execution across stages. *throttLL'eM* is orthogonal to DDP, used by [44], [46] to deploy multiple LLM instances.

throttLL'eM overheads: The proposed solution employs a generation length prediction model. Previous works [51] show that such models introduce an average latency of 7.6ms, which is negligible when compared to the multiple seconds of the inference process. throttLL'eM also requires profiling to generate the performance prediction model: a process that takes up to a day for LLaMa3-70B. However, this is a one-time cost that can be amortized over the lifetime of the service and across multiple instances. The prediction model requires less than a second of training on a multi-core CPU, while inference takes on average 3ms. Overall, when throttLL'eM is deployed, the *Scheduler* and *frequency throttling* components add 35ms to i) forecast KV cache utilization and batch sizes (2ms), ii) check multiple frequencies and schedule a query. Finally, setting a new frequency requires 8ms per GPU, which however occurs after request scheduling, off the critical path.

V. EVALUATION

A. Experimental Setup

HW/SW stack: We deploy *throttLL'eM* on a p4d.24xlarge AWS EC2 instance equipped with 8× NVIDIA A100 40GB GPUs [5]. For the software stack, we use NVIDIA's Triton server [43] and TensorRT-LLM backend [42] which provides high-performance LLM inference on NVIDIA GPUs. We enable typical LLM performance optimizations, such as: i) inflight fused batching [71]; ii) paged KV caching [31]; and iii) Flash Attention [17].

Examined LLMs and SLO definition: We examine LLMs of different sizes and configurations from the LLaMa family [60], presented in Table II. The TBT SLO is set as to an average latency below 200 ms, corresponding to the average human reading speed of 250 words per minute [8], a value adopted as a target constraint by MLPerf [39], [52]. To define the E2E SLO, we profile the LLMs at maximum GPU frequency using the MLPerf benchmark [52], gradually increasing the RPS until reaching saturation, characterized by long tail latencies in response times. The E2E SLO is set to the p99th response time under maximum load. Table II summarizes this information.

Load generation: Workload is generated using invocation traces from Azure [46], analyzed in Sec. III-D. Since the trace has a peak RPS of ≈ 8.25 , we right-scale the invocation rate to match the maximum load of the respective evaluated engine (reported in Table II). As the trace does not provide the actual context of input and output due to data protection regulations, we generate synthetic queries that match the prompt and generation lengths of each item in the dataset.

TABLE II: Performance profiles of examined LLMs.

	TP (#)	max load (RPS)	E2E SLO (p99 th)	KV blocks (#)
Llama3-8B	1	13.0	37.7	1033
Llama2-13B	1	1.125	22.7	120
Llama2-13B	2	4.0	30.2	439
Llama2-13B	4	7.5	31.3	1050
Llama3-70B	8	7.0	44.0	2205

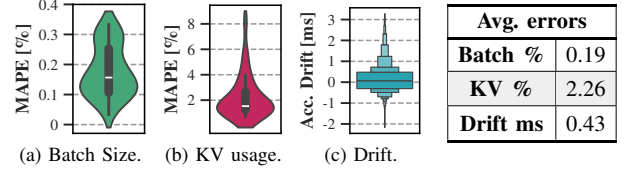


Fig. 7: Distributions and average values of various metrics

B. Performance Prediction Model Evaluation

Table III reports the R^2 score, *Mean Absolute Error* (MAE) and *Mean Absolute Percentage Error* (MAPE) of the performance prediction model ($\mathcal{M}$) employed by *throttLL'eM* (Sec. IV-C1), for each one of the examined engines and under two different train/test splits. Using a 90/10 split, $\mathcal{M}$ accurately predicts target IPS with R^2 scores ≥ 0.97 , often exceeding 0.98. MAPE is kept below 5.8% for all configurations, reaching a low of 2.8% for Llama2-13B when running on a single GPU. MAE, providing a more tangible representation of the results, indicates mispredictions under 1 IPS on average for all configurations. With a 10/90 train/test split, prediction performance remains robust, featuring R^2 scores above 0.96, MAE below 1.01 IPS, and a MAPE increase of 0.7% at most. These results demonstrate that high accuracy is achievable even with sparse training sets.

C. KV cache and Batch size Projection Mechanism Evaluation

We evaluate the *KV cache usage and batch size projection* component (Sec. IV-B) using custom generated micro-traces. Each micro-trace sets a different GPU frequency and includes queries with random prompt and generation lengths. We spawn all queries simultaneously which trigger *throttLL'eM*'s projection and prediction mechanisms, generating vectors KV , B and $\hat{T}_R$ for the specified frequency, which contains the estimated remaining time to reach all future iterations. At the same time, the performance monitoring endpoint of Triton is continuously polled, logging the actual batch size, KV cache usage and arrival time at each iteration. Figures 7a, 7b show the error distributions between actual and predicted batch size and KV cache usage respectively, while Fig. 7c plots the accumulated drift, i.e., how much predicted times (elements of $\hat{T}_R$) diverge from actual arrival times, divided by the number of elapsed iterations. Overall, batch size projection errors are kept to a minimum, averaging an error of 0.19%. The KV projection error is slightly higher while still remaining low at 2.26% on average. These small variations are attributed to the latency experienced between request submission and admission, during which execution progresses, occasionally resulting in off-by-one errors when calculating the final iteration of each request. Finally, each LLM iteration introduces a relatively small average drift of 0.43 ms, compared to the TBT (15 to 30 ms) for the same inference engine, shown in Fig. 2c. This drift is a combined effect of i) the aforementioned errors in KV cache usage and Batch size projections; and ii) estimation errors of the performance prediction model $\mathcal{M}$.

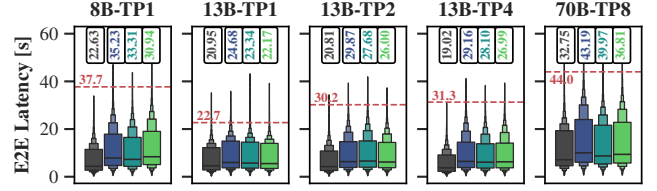
D. *throttLL'eM* end-to-end Evaluation

1) *throttLL'eM* without autoscaling: We first evaluate *throttLL'eM* without autoscaling, under the assumption of i) an oracle generation length predictor (predicted length $\widehat{r}_a$ equals actual generation length) and ii) error-prone predictors with 15% and 30% p95th error, reflecting typical prediction errors reported in the literature [23], [27], [51]. To simulate prediction errors for the known-length queries of the Azure trace, we add Gaussian noise to the reported lengths using σ values corresponding to 15% and 30% p95th prediction errors.

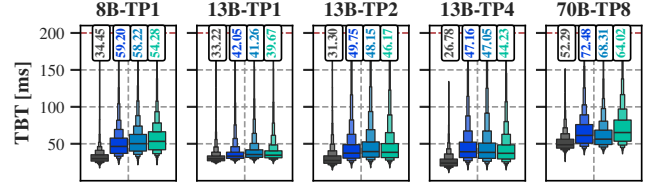
Figure 8a shows the distribution of E2E latencies for different engines when Triton [43] and *throttLL'eM* are deployed on the scaled trace. In almost all cases, with the exception of llama2-13b-TP1, *throttLL'eM* achieves the specified p99th latency SLO, being on average 1.45 seconds under the deadline. We attribute the inability of *throttLL'eM* to achieve the E2E SLO for llama2-13b-TP1 to two factors. First, even the default Triton implementation stays just 1.7 seconds below the SLO, hinting that *throttling* may not be applicable to this situation. Secondly, the very limited number of KV blocks available (120 per Tab. II) means that i) the engine is limited to lower batch size and throughput and ii) requests are often queued to avoid swapping to system memory. The TBT SLO is achieved in all cases, as depicted in Fig. 8b, with the vast majority of TBT values falling well below 200 ms. Outlier TBT values can be attributed to *inflight batching* overheads, as token generation stalls until the prefill phase of a new request is completed.

throttLL'eM aims to increase the energy efficiency of LLM serving. Figure 8c examines the distribution of power consumption values encountered by Triton and *throttLL'eM* under different prediction error levels. Triton displays consistently high power draw for all LLM engines. On the other hand, the proposed solution offers reduced power consumption due to lower GPU frequencies. As *throttLL'eM* dynamically adjusts frequencies to meet SLOs, a wide range of power values is encountered, including peak power equal to that of Triton when under high system pressure. Overall, *throttLL'eM* offers 36.3% higher energy efficiency on average with oracle predictions, dropping to 30.0% at 30% prediction error level. The highest increase in energy efficiency was experienced for llama2-13b-TP2, producing up to 44.3% more tokens per Joule than Triton. Overall, energy consumption is reduced by an average of 24.7% and up to 30.7%.

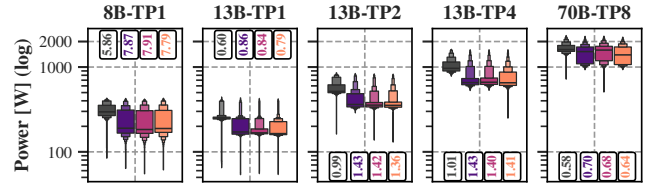
These improvements are enabled by the selection of lower frequencies. The barplots of Figure 9, show that *throttLL'eM*



(a) End-to-End latency distribution. The red dotted line denotes the E2E SLO of each model as defined in Table II. Text boxes report the 99th E2E latency percentile achieved by the different approaches.



(b) TBT distribution. The red dotted line denotes the TBT SLO (200 ms average latency between token generation). Text boxes report the average TBT latency achieved by the different approaches.



(c) Power Distribution (log scale). Text boxes report the energy efficiency (mean TPJ) achieved by the different approaches.

Fig. 8: Comparison of E2E, TBT and Power consumption between Triton [43] and *throttLL'eM*. At each plot, the leftmost box represents Triton, followed by *throttLL'eM* with error levels of 0%, 15%, and 30%, respectively.

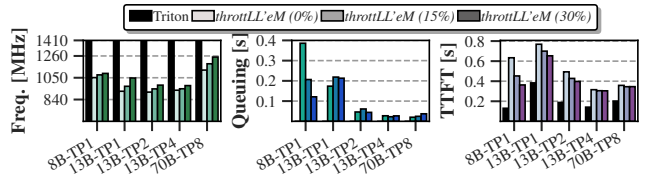


Fig. 9: Average GPU frequencies, Queue times and TTFT for various LLMs and configurations.

selects lower, more efficient frequencies than the default 1410 MHz, ranging between 950 and 1260 MHz on average, depending on the LLM engine and the prediction error of the predictor. Higher prediction errors lead to the selection of higher frequencies, as *throttLL'eM* increases the predicted generation length, leading to frequency overprovisioning.

Finally, we analyze the overheads imposed by the introduction of our framework. *throttLL'eM* maintains a queue where queries are held until the SLOs are validated. This introduces additional latency due to the time needed to perform the validation and the possible re-queuing of a request if it is predicted to cause violations. Figure 9 contains representative examples for both situations. Llama3-8b-TP1 exhibits

TABLE III: Performance Prediction Model Evaluation

	train = 90%			train = 10%		
	R ² (-)	MAPE (%)	MAE (iters/s)	R ² (-)	MAPE (%)	MAE (iters/s)
Llama3-8B-TP1	0.99	4.1	0.85	0.98	4.2	0.93
Llama2-13B-TP1	0.98	2.8	0.74	0.97	3.0	0.79
Llama2-13B-TP2	0.99	3.1	0.90	0.99	3.4	0.99
Llama2-13B-TP4	0.99	3.3	0.97	0.99	3.4	1.01
Llama3-70B-TP8	0.97	5.8	0.69	0.96	6.5	0.77

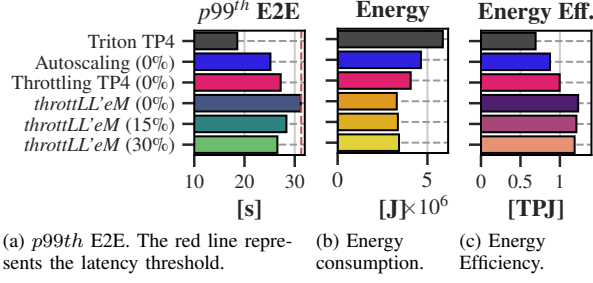


Fig. 10: Comparison of various implementations on the trace.

relatively high queuing delays as high request arrival rates stress the *Scheduler*, which can only schedule one request at a time. Llama2-13b-TP1 also exhibits delays, as the limited KV capacity leads to queuing until sufficient blocks become available. From the user’s perspective queuing delays add to the latency of the prefill phase (TTFT). Figure 9 shows that *throttLL'eM* increases TTFT when compared to Triton. A significant part of this increase is attributed to queuing, while the rest is due to the lower frequencies selected, which slow down the compute-bound prefill phase [4].

2) *throttLL'eM* with autoscaling: Last, we evaluate *throttLL'eM* including autoscaling using Llama2-13B’s TP1, TP2 and TP4 configurations. To ensure that all configurations are encountered, we scale the RPS range to $[0.75, 7.5]$ (up to the max load of TP4) by applying different scaling factors to different areas of the trace, thus keeping its shape, but amplifying variations between highest and lowest RPS.

To quantify the improvements afforded by both throttling and autoscaling, we compare *throttLL'eM* to: i) the default Triton implementation on the TP4 engine; ii) Triton equipped with *throttLL'eM* autoscaling; iii) *throttLL'eM* without autoscaling (TP4 only) and oracle predictions. These cover the entire range of autoscaling and throttling combinations. Our solution is again evaluated using multiple prediction error levels. Figures 10a and 10b show the E2E latency and energy consumption measured during the execution of the trace for all implementations. As expected, Triton features the lowest E2E latency but also the highest energy consumption. Enabling autoscaling (throttling) increases E2E latency as some performance is sacrificed for a 20.8% (30.6%) reduction in energy consumption. Using both, *throttLL'eM* achieves the lowest energy consumption, 43.8% (41.7%) lower than Triton at 0% (30%) error level, but also the highest E2E latency, which however remains below the $p99^{th}$ threshold value for the TP4 engine. TBT SLOs are met in all cases (avg. TBTs <50ms, not shown). From an energy efficiency standpoint (Fig. 10c), Triton on TP4 achieves 0.69 TPJ while autoscaling-only and throttling-only achieve 0.87 and 0.99 TPJ respectively. *throttLL'eM* which implements both achieves 1.19 to 1.23 TPJ depending on the error of the predictor, a $1.71\times$ to $1.78\times$ increase in efficiency over the baseline implementation.

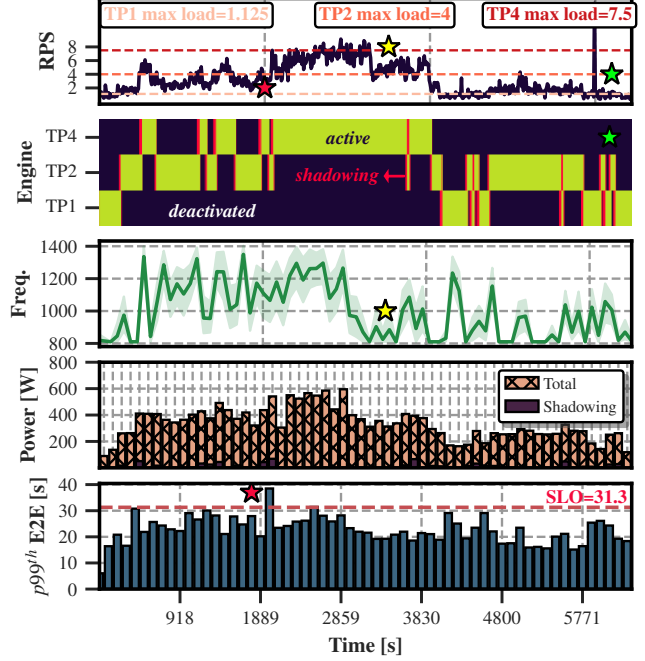


Fig. 11: Runtime Analysis of *throttLL'eM* on the scaled trace.

E. *throttLL'eM* Runtime Analysis

This section analyzes how *throttLL'eM* adapts under varying load. Figure 11 shows: i) experienced RPS; ii) the state of different TP engines; iii) applied frequencies; iv) average power draw; and v) $p99^{th}$ E2E latency over time. Different colors in the *Engine* plot denote various LLM engine states. Average power is shown using hatched orange bars, while the power draw attributed to shadowing is shown in solid purple.

In general, *throttLL'eM* selects lower frequencies when the *active* engine is under light load. When however approaching the maximum load of an active engine (max load in Table II), frequency is increased above the energy efficiency “sweet spot” to prevent SLO violations.

Breaking down the experienced $p99^{th}$ E2E latency to individual sections, it becomes evident that SLO violations may only be encountered when i) operating above the maximum load of the largest (TP4) engine or ii) during sudden increases in RPS, until up-scaling is completed. The former exceeds the rated capacity of the system, while the latter cannot be resolved unless startup times are reduced. Despite these transient violations, *throttLL'eM* manages to achieve the $p99^{th}$ E2E SLO over the full duration of the trace.

Figure 11 features some points of interest. The red stars mark transient E2E SLO violations due to a sudden load increase that cannot be absorbed by the TP2 engine while TP4 comes online. Yellow stars highlight how frequency is reduced when the instance is operating at lower load but cannot be downsized. Finally, green stars mark consecutive switches, as the encountered load remains close to the TP1/TP2 engine switching load boundary of 1.125 RPS.

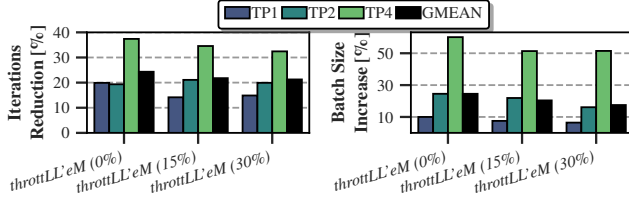


Fig. 12: **a)** Reduction in iterations and **b)** Increase in avg. batch size of throttLL'eM to autoscale-only for all error levels.

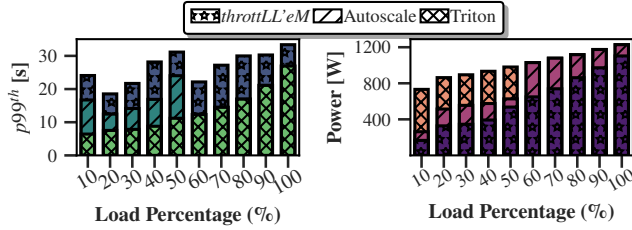


Fig. 13: Comparison of **a)** $p99^{th}$ E2E and **b)** avg. Power for different implementations at increasing system load levels.

★ **Takeaways:** Autoscaling performs coarse-grained optimization by right-sizing system capacity. GPU throttling performs finer adjustments that cannot be achieved by autoscaling.

F. Extended Evaluation & Comparison

1) *Sources of Energy Gains:* The observed energy gains in throttLL'eM are driven by two key mechanisms. First, autoscaling adjusts the allocated capacity, resulting in a 20.8% reduction in energy consumption compared to Triton TP4, as fewer GPUs are required, lowering static power draw across the entire trace. Second, frequency scaling further reduces energy consumption by operating GPUs at lower frequencies, as shown in Sec. III-A1. A side effect of this is that lower frequencies slow down the inference process, allowing more requests to accumulate in each batch, thereby artificially increasing batch size. This improves energy efficiency (cf. Fig. 2e), but also decreases the number of performed LLM iterations, leveraging the higher batch-level parallelism to generate more tokens per forward pass. Figure 12 illustrates this behaviour, showing how throttLL'eM increases batch size and reduces iterations compared to autoscaling alone. At 0% predictor error level, throttLL'eM manages to increase batch size by a geo-mean of 24.5% across all engines, performing 24.3% less iterations. Although higher errors result in higher frequencies, which diminish these benefits, the overall trend remains consistent.

2) *throttLL'eM operation under varying loads:* We examine the trade-off between energy efficiency and latency by progressively increasing the system load from 10% to 100% across different implementations. Figure 13b demonstrates how throttLL'eM is able to reduce average power compared to Triton and autoscaling-only by downsizing engines and reducing frequencies. In this manner, throttLL'eM trades-off latency, which as shown in Fig. 13a, is brought close to that

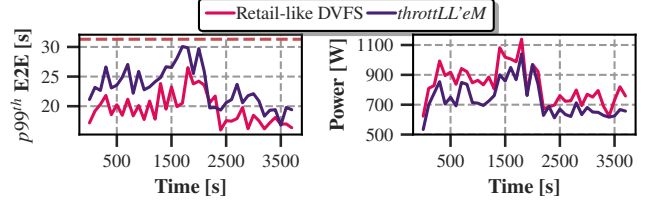


Fig. 14: **a)** $p99^{th}$ E2E and **b)** Avg. power draw of throttLL'eM (w/o autoscaling) and a Retail-inspired implementation.

of peak load (31.3s per Tab. II). While autoscaling-only can significantly reduce the average power draw, e.g., at 10% load level, in other cases the frequency scaling logic of throttLL'eM offers additional gains which cannot be achieved with coarse-grained optimizations like autoscaling.

3) *Comparison with other DVFS schemes:* To demonstrate the effectiveness of our LLM-aware design choices, we compare throttLL'eM with a SotA microservice-based DVFS scheme, similar to ReTail [10]. ReTail automatically identifies input and application features that strongly correlate to E2E latency, in our case: frequency, batch size, KV cache utilization and input/output length. Using these, it trains a *linear regression* model to predict latency and selects the lowest frequency that satisfies the SLOs for all requests in the queue. Upon receiving a new request, ReTail evaluates available frequencies and chooses the minimum that meets SLO requirements.

To ensure a fair comparison, we used the LLaMa2-13B model in its TP4 configuration, as ReTail does not support autoscaling. Both implementations were tested at a 30% predictor error level. Figure 14b shows that throttLL'eM consistently achieves lower power consumption, while Fig. 14a demonstrates its ability to more aggressively approach the latency deadlines. We attribute this to i) throttLL'eM's capability to forecast KV cache utilization and batch size, and ii) a more accurate ML-based performance prediction model. In detail, ReTail relies on the currently known values of KV cache utilization and batch size for latency prediction, which however change over time, whereas throttLL'eM uses forecasted values for future iterations. Moreover, our analysis showed that ReTail's linear regression model struggles with E2E latency prediction, achieving R^2 close to 0.4. This is due to the non-linear E2E latency slowdown, as token generation proceeds. throttLL'eM addresses this by using iteration-level predictions which are then aggregated to produce accurate E2E latency predictions. Overall, across the entire trace throttLL'eM consumes 11.9% lower energy, a value that would have been higher had autoscaling also been enabled.

4) *Evaluation on different systems/LLMs:* We further examine how throttLL'eM adapts to diverse systems and LLMs, by deploying a Mixtral 8x7B Mixture-of-Experts LLM [25] on a g5.48xlarge AWS instance with 8x NVIDIA A10G 24GB GPUs. This system features lower performance than the previously examined one, lacking the HBM2 memory and the 600 GB/s NVLink interconnects of p4d.24xlarge. After

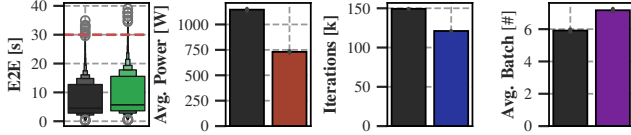


Fig. 15: **a)** E2E latency **b)** Total Energy Consumption **c)** Total iterations and **d)** Average Batch size of Triton (left) and *throttLL'eM* (right) of Mixtral 8x7B on NVIDIA A10G (TP8).

profiling and training, $\mathcal{M}$ features an R^2 (MAPE) score of 0.98 (5.8%), displaying accurate modelling performance. We scale the trace at the peak load of 1.25 RPS and a target latency of 30s. Figure 15 displays the $p99^{th}$ E2E latencies and the average power draw of both implementations when serving the scaled trace. In this setup, *throttLL'eM* is able to reduce average power by 35.9%, while still featuring a $p99^{th}$ E2E latency of 28.4s, being well within the 30s SLO for this configuration.

VI. RELATED WORK

LLM serving optimization is an active research topic, with many works aiming to improve the inference process [80].

LLM inference optimization: Recent research has explored various optimization techniques for LLM inference. These efforts address different facets of LLM inference, including i) efficient request batching [20], [71] with *inflight batching* introduced by Orca [71] being a pioneering work in the field, ii) efficient LLM serving and request scheduling [4], [23], [35], [38], [43], [46], [55], [57], [63], [66], [71], [78] with NVIDIA's Triton [43] being a state-of-the-art solution for inference serving offering optimized performance, iii) LLM memory optimization [20], [31], [56], [76] with *paged attention* [31] being a notable innovation in the area, iv) model scaling and parallelism on multiple GPUs [6], [26], [49], v) speculative decoding and sampling [9], [33], [37], [58], [67], [68], vi) parameter sharing [79] and vii) optimization of GPU kernels [2], [6], [16], [17], [62], [72] with Flash Attention [16], [17] and NVIDIA's FasterTransformer [2] being the most notable works. The work presented in [47] manages multiple queues for interactive and batched inference requests, avoiding potential head-of-line blocking. While these works primarily target performance optimization, neglecting energy efficiency, they offer orthogonal techniques that can complement *throttLL'eM*. Notably, as mentioned in Sec. V-A, *throttLL'eM* is built on top of Triton [43] which integrates optimizations like *inflight batching* [71], *paged attention* [31] and *flash attention* [16].

Power and Energy Efficient LLM inference serving: Recent works explore the energy and power implications of LLM inference. Studies such as [12], [53], [59] characterize energy consumption and carbon emissions but lack frameworks for energy-efficient LLM inference. The authors of [65] model the performance and energy consumption of LLM queries using an analytical approach. They use this model to solve an offline scheduling optimization problem. However, they

overlook the dynamic nature of inference systems and KV cache implications (cf. Sec. III-B). In [28] a framework for energy-efficient LLM inference is proposed, but is limited to fixed-length queries (`max_tokens`) on single-GPU models. A concurrent work, μ -Serve [50] performs intelligent model partition placement and introduces a multiplicative-increase-additive-decrease (MIAD)-based DVFS scheme to reduce power consumption while meeting the SLOs of LLM requests based on their predicted generation length and SLO slack. Lastly, [44] presents POLCA, a framework for power oversubscription in LLM clusters. *throttLL'eM* can complement POLCA by performing frequency scaling under power-capping constraints, enhancing the margins for power oversubscription.

VII. CONCLUSION

LLM inference presents an emerging workload with considerable energy footprint. In this work we performed an extensive study on the factors that affect the inference process, gathering insights on performance and energy efficiency. Guided by these insights, we designed *throttLL'eM*, a framework that uses a predictive, ML-driven approach to increase the energy efficiency of LLM inference serving under SLOs. *throttLL'eM* improves efficiency by over $1.71\times$, consuming up to 43.8% less energy than Triton.

ACKNOWLEDGEMENTS

The authors would like to thank the National Infrastructure for Research and Technology Network (GRNET) for funding the cloud infrastructure used to evaluate *throttLL'eM*. They further acknowledge Georgios Zervakis and Panagiotis Hadji-doukas for granting access to the DGX Deep Learning System of the Computer Engineering and Informatics Department (CEID), University of Patras, utilized during development. Lastly, they are grateful to Konstantinos Kanellopoulos and the anonymous reviewers for their constructive feedback, which enhanced the quality of this paper.

This work has been partially funded by the PRIVATEER project. PRIVATEER has received funding from the Smart Networks and Services Joint Undertaking (SNS JU) under the European Union's Horizon Europe research and innovation programme under Grant Agreement No. 101096110. Views and opinions expressed are however those of the author(s) only and do not necessarily reflect those of the European Union or SNS JU. Neither the European Union nor the granting authority can be held responsible for them.

REFERENCES

- [1] "Exclusive: ChatGPT traffic slips again for third month in a row," <https://www.reuters.com/technology/chatgpt-traffic-slips-again-third-month-row-2023-09-07/>.
- [2] "NVIDIA FasterTransformer," <https://github.com/NVIDIA/FasterTransformer>.
- [3] J. Achiam, S. Adler, S. Agarwal, L. Ahmad, I. Akkaya, F. L. Aleman, D. Almeida, J. Altenschmidt, S. Altman, S. Anadkat *et al.*, "Gpt-4 technical report," *arXiv preprint arXiv:2303.08774*, 2023.

- [4] A. Agrawal, N. Kedia, A. Panwar, J. Mohan, N. Kwatra, B. Gulavani, A. Tumanov, and R. Ramjee, "Taming {Throughput-Latency} tradeoff in {LLM} inference with {Sarathi-Serve}," in *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 24)*, 2024, pp. 117–134.
- [5] Amazon AWS, "Amazon ec2 p4 instances." <https://aws.amazon.com/ec2/instance-types/p4/>.
- [6] R. Y. Aminabadi, S. Rajbhandari, A. A. Awan, C. Li, D. Li, E. Zheng, O. Ruwase, S. Smith, M. Zhang, J. Rasley *et al.*, "DeepSpeed-inference: enabling efficient inference of transformer models at unprecedented scale," in *SC22: International Conference for High Performance Computing, Networking, Storage and Analysis*. IEEE, 2022, pp. 1–15.
- [7] Anna-Sofia Lesiv, "How much energy will it take to power ai?" <https://www.contrary.com/foundations-and-frontiers/ai-inference>.
- [8] M. Brysbaert, "How many words do we read per minute? a review and meta-analysis of reading rate," *Journal of memory and language*, vol. 109, p. 104047, 2019.
- [9] C. Chen, S. Borgeaud, G. Irving, J.-B. Lespiau, L. Sifre, and J. Junger, "Accelerating large language model decoding with speculative sampling," *arXiv preprint arXiv:2302.01318*, 2023.
- [10] S. Chen, A. Jin, C. Delimitrou, and J. F. Martínez, "Retail: Opting for learning simplicity to enable qos-aware power management in the cloud," in *2022 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*, 2022, pp. 155–168.
- [11] T. Chen and C. Guestrin, "Xgboost: A scalable tree boosting system," in *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, ser. KDD '16. New York, NY, USA: Association for Computing Machinery, 2016, p. 785–794. [Online]. Available: <https://doi.org/10.1145/2939672.2939785>
- [12] A. A. Chien, L. Lin, H. Nguyen, V. Rao, T. Sharma, and R. Wijayaradana, "Reducing the carbon impact of generative ai inference (today and in 2035)," in *Proceedings of the 2nd workshop on sustainable computer systems*, 2023, pp. 1–7.
- [13] M. Chow, A. Jahanshahi, and D. Wong, "Krisp: Enabling kernel-wise right-sizing for spatial partitioned gpu inference servers," in *2023 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*, 2023, pp. 624–637.
- [14] L. Contributors, "Lmdeploy: A toolkit for compressing, deploying, and serving llm," <https://github.com/InternLM/lmdeploy>, 2023.
- [15] D. Crankshaw, X. Wang, G. Zhou, M. J. Franklin, J. E. Gonzalez, and I. Stoica, "Clipper: A low-latency online prediction serving system," in *14th USENIX Symposium on Networked Systems Design and Implementation, NSDI 2017, Boston, MA, USA, March 27-29, 2017*, A. Akella and J. Howell, Eds. USENIX Association, 2017, pp. 613–627. [Online]. Available: <https://www.usenix.org/conference/nsdi17/technical-sessions/presentation/crankshaw>
- [16] T. Dao, "Flashattention-2: Faster attention with better parallelism and work partitioning," *arXiv preprint arXiv:2307.08691*, 2023.
- [17] T. Dao, D. Y. Fu, S. Ermon, A. Rudra, and C. Ré, "Flashattention: Fast and memory-efficient exact attention with io-awareness," 2022.
- [18] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "Bert: Pre-training of deep bidirectional transformers for language understanding," *arXiv preprint arXiv:1810.04805*, 2018.
- [19] A. Dhakal, S. G. Kulkarni, and K. Ramakrishnan, "Gslice: controlled spatial sharing of gpus for a scalable inference platform," in *Proceedings of the 11th ACM Symposium on Cloud Computing*, 2020, pp. 492–506.
- [20] J. Fang, Y. Yu, C. Zhao, and J. Zhou, "Turbotransformers: an efficient gpu serving system for transformer models," in *Proceedings of the 26th ACM SIGPLAN Symposium on Principles and Practice of Parallel Programming*, 2021, pp. 389–402.
- [21] J. H. Friedman, "Greedy function approximation: a gradient boosting machine," *Annals of statistics*, pp. 1189–1232, 2001.
- [22] U. Gupta, Y. G. Kim, S. Lee, J. Tse, H.-H. S. Lee, G.-Y. Wei, D. Brooks, and C.-J. Wu, "Chasing carbon: The elusive environmental footprint of computing," *IEEE Micro*, vol. 42, no. 4, p. 37–47, Jul. 2022. [Online]. Available: <https://doi.org/10.1109/MM.2022.3163226>
- [23] C. Hu, H. Huang, L. Xu, X. Chen, J. Xu, S. Chen, H. Feng, C. Wang, S. Wang, Y. Bao *et al.*, "Inference without interference: Disaggregate llm inference for mixed downstream workloads," *arXiv preprint arXiv:2401.11181*, 2024.
- [24] IEA, Paris, "Iea 2024," <https://www.iea.org/reports/electricity-2024>, Licence: CC BY 4.0.
- [25] A. Q. Jiang, A. Sablayrolles, A. Roux, A. Mensch, B. Savary, C. Bamford, D. S. Chaplot, D. d. l. Casas, E. B. Hanna, F. Bressand *et al.*, "Mixtral of experts," *arXiv preprint arXiv:2401.04088*, 2024.
- [26] Z. Jiang, H. Lin, Y. Zhong, Q. Huang, Y. Chen, Z. Zhang, Y. Peng, X. Li, C. Xie, S. Nong, Y. Jia, S. He, H. Chen, Z. Bai, Q. Hou, S. Yan, D. Zhou, Y. Sheng, Z. Jiang, H. Xu, H. Wei, Z. Zhang, P. Nie, L. Zou, S. Zhao, L. Xiang, Z. Liu, Z. Li, X. Jia, J. Ye, X. Jin, and X. Liu, "MegaScale: Scaling large language model training to more than 10,000 GPUs," in *21st USENIX Symposium on Networked Systems Design and Implementation (NSDI 24)*. Santa Clara, CA: USENIX Association, Apr. 2024, pp. 745–760. [Online]. Available: <https://www.usenix.org/conference/nsdi24/presentation/jiang-ziheng>
- [27] Y. Jin, C.-F. Wu, D. Brooks, and G.-Y. Wei, "S³: Increasing gpu utilization during generative inference for higher throughput," 2023.
- [28] A. Kakolyris, D. Masouros, S. Xydis, and D. Soudris, "Slo-aware gpu dvfs for energy-efficient llm inference serving," *IEEE Computer Architecture Letters*, no. 01, pp. 1–4, may 5555.
- [29] D. H. Kim, C. Imes, and H. Hoffmann, "Racing and pacing to idle: Theoretical and empirical analysis of energy optimization heuristics," in *2015 IEEE 3rd international conference on cyber-physical systems, networks, and applications*. IEEE, 2015, pp. 78–85.
- [30] A. G. Kumbhare, R. Azimi, I. Manousakis, A. Bonde, F. V. Frujeri, N. Mahalingam, P. A. Misra, S. A. Javadi, B. Schroeder, M. Fontoura, and R. Bianchini, "Prediction-based power oversubscription in cloud platforms," in *2021 USENIX Annual Technical Conference, USENIX ATC 2021, July 14-16, 2021*, I. Calciu and G. Kuenning, Eds. USENIX Association, 2021, pp. 473–487. [Online]. Available: <https://www.usenix.org/conference/atc21/presentation/kumbhare>
- [31] W. Kwon, Z. Li, S. Zhuang, Y. Sheng, L. Zheng, C. H. Yu, J. E. Gonzalez, H. Zhang, and I. Stoica, "Efficient memory management for large language model serving with pagedattention," 2023.
- [32] E. Le Sueur and G. Heiser, "Slow down or sleep, that is the question," in *2011 USENIX Annual Technical Conference (USENIX ATC 11)*, 2011.
- [33] Y. Leviathan, M. Kalman, and Y. Matias, "Fast inference from transformers via speculative decoding," in *International Conference on Machine Learning*. PMLR, 2023, pp. 19 274–19 286.
- [34] S. Li, X. Wang, X. Zhang, V. Kontorinis, S. Kodakara, D. Lo, and P. Ranganathan, "Thunderbolt: Throughput-optimized, quality-of-service-aware power capping at scale," in *14th USENIX Symposium on Operating Systems Design and Implementation, OSDI 2020, Virtual Event, November 4-6, 2020*. USENIX Association, 2020, pp. 1241–1255. [Online]. Available: <https://www.usenix.org/conference/osdi20/presentation/li-shaohong>
- [35] Z. Li, L. Zheng, Y. Zhong, V. Liu, Y. Sheng, X. Jin, Y. Huang, Z. Chen, H. Zhang, J. E. Gonzalez *et al.*, "{AlpaServe}: Statistical multiplexing with model parallelism for deep learning serving," in *17th USENIX Symposium on Operating Systems Design and Implementation (OSDI 23)*, 2023, pp. 663–679.
- [36] A. Malistov and A. Trushin, "Gradient boosted trees with extrapolation," in *2019 18th IEEE International Conference On Machine Learning And Applications (ICMLA)*. IEEE, 2019, pp. 783–789.
- [37] X. Miao, G. Oliaro, Z. Zhang, X. Cheng, Z. Wang, Z. Zhang, R. Y. Y. Wong, A. Zhu, L. Yang, X. Shi *et al.*, "Specinfer: Accelerating large language model serving with tree-based speculative inference and verification," in *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 3*, 2024, pp. 932–949.
- [38] X. Miao, C. Shi, J. Duan, X. Xi, D. Lin, B. Cui, and Z. Jia, "Spotserve: Serving generative large language models on preemptible instances," in *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2*, ser. ASPLOS '24. New York, NY, USA: Association for Computing Machinery, 2024, p. 1112–1127. [Online]. Available: <https://doi.org/10.1145/3620665.3640411>
- [39] ML Commons, "Llama 2 70b: An mlperf inference benchmark for large language models," <https://mlcommons.org/2024/03/mlperf-llama2-70b/>.
- [40] S. M. Nabavinejad, S. Reda, and M. Ebrahimi, "Coordinated batching and DVFS for DNN inference on GPU accelerators," *IEEE Trans. Parallel Distributed Syst.*, vol. 33, no. 10, pp. 2496–2508, 2022. [Online]. Available: <https://doi.org/10.1109/TPDS.2022.3144614>
- [41] NVIDIA Corporation, "Parallelisms," <https://docs.nvidia.com/nemo-framework/user-guide/latest/nemotoolkit/features/parallelisms.html>.
- [42] NVIDIA Corporation, "Tensortt-llm backend: The triton backend

- for tensorrt-llm.” https://github.com/triton-inference-server/tensorrtllm_backend.
- [43] NVIDIA Corporation, “Triton inference server,” <https://github.com/triton-inference-server>.
- [44] P. Patel, E. Choukse, C. Zhang, I. n. Goiri, B. Warriar, N. Mahalingam, and R. Bianchini, “Characterizing power management opportunities for llms in the cloud,” in *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 3*, ser. ASPLOS ’24. New York, NY, USA: Association for Computing Machinery, 2024, p. 207–222. [Online]. Available: <https://doi.org/10.1145/3620666.3651329>
- [45] P. Patel, E. Choukse, C. Zhang, I. Goiri, B. Warriar, N. Mahalingam, and R. Bianchini, “Characterizing power management opportunities for llms in the cloud,” in *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 3*, 2024, pp. 207–222.
- [46] P. Patel, E. Choukse, C. Zhang, A. Shah, I. Goiri, S. Maleki, and R. Bianchini, “Splitwise: Efficient generative llm inference using phase splitting,” in *ISCA*, June 2024.
- [47] A. Patke, D. Reddy, S. Jha, H. Qiu, C. Pinto, C. Narayanaswami, Z. Kalbarczyk, and R. Iyer, “Queue management for slo-oriented large language model serving,” in *Proceedings of the 2024 ACM Symposium on Cloud Computing*, ser. SoCC ’24. New York, NY, USA: Association for Computing Machinery, 2024, p. 18–35. [Online]. Available: <https://doi.org/10.1145/3698038.3698523>
- [48] D. Patterson, J. Gonzalez, U. Hölzle, Q. Le, C. Liang, L.-M. Munguia, D. Rothchild, D. R. So, M. Texier, and J. Dean, “The carbon footprint of machine learning training will plateau, then shrink,” *Computer*, vol. 55, no. 7, pp. 18–28, 2022.
- [49] R. Pope, S. Douglas, A. Chowdhery, J. Devlin, J. Bradbury, J. Heek, K. Xiao, S. Agrawal, and J. Dean, “Efficiently scaling transformer inference,” *Proceedings of Machine Learning and Systems*, vol. 5, pp. 606–624, 2023.
- [50] H. Qiu, W. Mao, A. Patke, S. Cui, S. Jha, C. Wang, H. Franke, Z. Kalbarczyk, T. Başar, and R. K. Iyer, “Power-aware deep learning model serving with μ -serve,” in *2024 USENIX Annual Technical Conference (USENIX ATC 24)*. Santa Clara, CA: USENIX Association, Jul. 2024, pp. 75–93. [Online]. Available: <https://www.usenix.org/conference/atc24/presentation/qiu>
- [51] H. Qiu, W. Mao, A. Patke, S. Cui, S. Jha, C. Wang, H. Franke, Z. T. Kalbarczyk, T. Başar, and R. K. Iyer, “Efficient interactive llm serving with proxy model-based sequence length prediction,” *arXiv preprint arXiv:2404.08509*, 2024.
- [52] V. J. Reddi, C. Cheng, D. Kanter, P. Mattson, G. Schmuelling, C.-J. Wu, B. Anderson, M. Breughe, M. Charlebois, W. Chou, R. Chukka, C. Coleman, S. Davis, P. Deng, G. Damos, J. Duke, D. Fick, J. S. Gardner, I. Hubara, S. Idgunji, T. B. Jablin, J. Jiao, T. S. John, P. Kanwar, D. Lee, J. Liao, A. Lohmotov, F. Massa, P. Meng, P. Mickevicius, C. Osborne, G. Pekhimenko, A. T. R. Rajan, D. Sequeira, A. Sirasao, F. Sun, H. Tang, M. Thomson, F. Wei, E. Wu, L. Xu, K. Yamada, B. Yu, G. Yuan, A. Zhong, P. Zhang, and Y. Zhou, “Mlperf inference benchmark,” in *2020 ACM/IEEE 47th Annual International Symposium on Computer Architecture (ISCA)*, 2020, pp. 446–459.
- [53] S. Samsi, D. Zhao, J. McDonald, B. Li, A. Michaleas, M. Jones, W. Bergeron, J. Kepner, D. Tiwari, and V. Gadepally, “From words to watts: Benchmarking the energy costs of large language model inference,” in *IEEE High Performance Extreme Computing Conference, HPEC 2023, Boston, MA, USA, September 25-29, 2023*. IEEE, 2023, pp. 1–9. [Online]. Available: <https://doi.org/10.1109/HPEC58863.2023.10363447>
- [54] V. Sanh, L. Debut, J. Chaumond, and T. Wolf, “Distilbert, a distilled version of bert: smaller, faster, cheaper and lighter,” *arXiv preprint arXiv:1910.01108*, 2019.
- [55] Y. Sheng, S. Cao, D. Li, B. Zhu, Z. Li, D. Zhuo, J. E. Gonzalez, and I. Stoica, “Fairness in serving large language models,” in *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 24)*, 2024, pp. 965–988.
- [56] Y. Sheng, L. Zheng, B. Yuan, Z. Li, M. Ryabinin, B. Chen, P. Liang, C. Ré, I. Stoica, and C. Zhang, “Flexgen: High-throughput generative inference of large language models with a single gpu,” in *International Conference on Machine Learning*. PMLR, 2023, pp. 31 094–31 116.
- [57] Y. Song, Z. Mi, H. Xie, and H. Chen, “Powerinfer: Fast large language model serving with a consumer-grade gpu,” 2023. [Online]. Available: <https://arxiv.org/abs/2312.12456>
- [58] M. Stern, N. Shazeer, and J. Uszkoreit, “Blockwise parallel decoding for deep autoregressive models,” *Advances in Neural Information Processing Systems*, vol. 31, 2018.
- [59] J. Stojkovic, E. Choukse, C. Zhang, I. Goiri, and J. Torrellas, “Towards greener llms: Bringing energy-efficiency to the forefront of llm inference,” *arXiv preprint arXiv:2403.20306*, 2024.
- [60] H. Touvron *et al.*, “Llama: Open and efficient foundation language models,” *CoRR*, vol. abs/2302.13971, 2023. [Online]. Available: <https://doi.org/10.48550/arXiv.2302.13971>
- [61] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin, “Attention is all you need,” in *Advances in Neural Information Processing Systems 30: Annual Conference on Neural Information Processing Systems 2017, December 4-9, 2017, Long Beach, CA, USA*, I. Guyon, U. von Luxburg, S. Bengio, H. M. Wallach, R. Fergus, S. V. N. Vishwanathan, and R. Garnett, Eds., 2017, pp. 5998–6008. [Online]. Available: <https://proceedings.neurips.cc/paper/2017/hash/3f5ee243547dee91fbd053c1c4a845aa-Abstract.html>
- [62] X. Wang, Y. Xiong, Y. Wei, M. Wang, and L. Li, “LightSeq: A high performance inference library for transformers,” in *Proceedings of the 2021 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies: Industry Papers*, Y.-b. Kim, Y. Li, and O. Rambow, Eds. Online: Association for Computational Linguistics, Jun. 2021, pp. 113–120. [Online]. Available: <https://aclanthology.org/2021.naacl-industry.15>
- [63] Y. Wang, K. Chen, H. Tan, and K. Guo, “Tabi: An efficient multi-level inference system for large language models,” in *Proceedings of the Eighteenth European Conference on Computer Systems*, 2023, pp. 233–248.
- [64] Y. Wang, Y. Chen, Z. Li, X. Kang, Z. Tang, X. He, R. Guo, X. Wang, Q. Wang, A. C. Zhou, and X. Chu, “Burstgpt: A real-world workload dataset to optimize llm serving systems,” 2024. [Online]. Available: <https://arxiv.org/abs/2401.17644>
- [65] G. Wilkins, S. Keshav, and R. Mortier, “Offline energy-optimal llm serving: Workload-based energy models for llm inference on heterogeneous systems,” *arXiv preprint arXiv:2407.04014*, 2024.
- [66] B. Wu, Y. Zhong, Z. Zhang, G. Huang, X. Liu, and X. Jin, “Fast distributed inference serving for large language models,” *arXiv preprint arXiv:2305.05920*, 2023.
- [67] H. Xia, T. Ge, P. Wang, S.-Q. Chen, F. Wei, and Z. Sui, “Speculative decoding: Exploiting speculative execution for accelerating seq2seq generation,” in *Findings of the Association for Computational Linguistics: EMNLP 2023*, H. Bouamor, J. Pino, and K. Bali, Eds. Singapore: Association for Computational Linguistics, Dec. 2023, pp. 3909–3925. [Online]. Available: <https://aclanthology.org/2023.findings-emnlp.257>
- [68] H. Xia, Z. Yang, Q. Dong, P. Wang, Y. Li, T. Ge, T. Liu, W. Li, and Z. Sui, “Unlocking efficiency in large language model inference: A comprehensive survey of speculative decoding,” *arXiv preprint arXiv:2401.07851*, 2024.
- [69] D. Yang, W. Rang, and D. Cheng, “Mitigating stragglers in the decentralized training on heterogeneous clusters,” in *Proceedings of the 21st International Middleware Conference*, 2020, pp. 386–399.
- [70] Z. Ye, W. Gao, Q. Hu, P. Sun, X. Wang, Y. Luo, T. Zhang, and Y. Wen, “Deep learning workload scheduling in GPU datacenters: A survey,” *ACM Comput. Surv.*, vol. 56, no. 6, pp. 146:1–146:38, 2024. [Online]. Available: <https://doi.org/10.1145/3638757>
- [71] G.-I. Yu *et al.*, “Orca: A distributed serving system for Transformer-Based generative models,” in *16th USENIX Symposium on Operating Systems Design and Implementation (OSDI 22)*. Carlsbad, CA: USENIX Association, Jul. 2022, pp. 521–538. [Online]. Available: <https://www.usenix.org/conference/osdi22/presentation/yy>
- [72] Y. Zhai, C. Jiang, L. Wang, X. Jia, S. Zhang, Z. Chen, X. Liu, and Y. Zhu, “Bytetransformer: A high-performance transformer boosted for variable-length inputs,” in *2023 IEEE International Parallel and Distributed Processing Symposium (IPDPS)*. IEEE, 2023, pp. 344–355.
- [73] C. Zhang *et al.*, “Flex: High-availability datacenters with zero reserved power,” in *2021 ACM/IEEE 48th Annual International Symposium on Computer Architecture (ISCA)*. IEEE, 2021, pp. 319–332.
- [74] C. Zhang, M. Yu, W. Wang, and F. Yan, “Mark: Exploiting cloud services for cost-effective, slo-aware machine learning inference serving,” in *2019 USENIX Annual Technical Conference, USENIX ATC 2019, Renton, WA, USA, July 10-12, 2019*, D. Malkhi and D. Tsafir,

- Eds. USENIX Association, 2019, pp. 1049–1062. [Online]. Available: <https://www.usenix.org/conference/atc19/presentation/zhang-chengliang>
- [75] S. Zhang, S. Roller, N. Goyal, M. Artetxe, M. Chen, S. Chen, C. Dewan, M. Diab, X. Li, X. V. Lin, T. Mihaylov, M. Ott, S. Shleifer, K. Shuster, D. Simig, P. S. Koura, A. Sridhar, T. Wang, and L. Zettlemoyer, “Opt: Open pre-trained transformer language models,” 2022.
 - [76] Z. Zhang, Y. Sheng, T. Zhou, T. Chen, L. Zheng, R. Cai, Z. Song, Y. Tian, C. Ré, C. Barrett *et al.*, “H2o: Heavy-hitter oracle for efficient generative inference of large language models,” *Advances in Neural Information Processing Systems*, vol. 36, 2024.
 - [77] Z. Zheng, X. Ren, F. Xue, Y. Luo, X. Jiang, and Y. You, “Response length perception and sequence scheduling: An llm-empowered llm inference pipeline,” in *Advances in Neural Information Processing Systems*, A. Oh, T. Naumann, A. Globerson, K. Saenko, M. Hardt, and S. Levine, Eds., vol. 36. Curran Associates, Inc., 2023, pp. 65 517–65 530. [Online]. Available: https://proceedings.neurips.cc/paper_files/paper/2023/file/ce7ff3405c782f761fac7f849b41ae9a-Paper-Conference.pdf
 - [78] Y. Zhong, S. Liu, J. Chen, J. Hu, Y. Zhu, X. Liu, X. Jin, and H. Zhang, “Distserve: Disaggregating prefill and decoding for goodput-optimized large language model serving,” in *18th USENIX Symposium on Operating Systems Design and Implementation, OSDI 2024, Santa Clara, CA, USA, July 10-12, 2024*, A. Gavrilovska and D. B. Terry, Eds. USENIX Association, 2024, pp. 193–210. [Online]. Available: <https://www.usenix.org/conference/osdi24/presentation/zhong-yinmin>
 - [79] Z. Zhou, X. Wei, J. Zhang, and G. Sun, “{PetS}: A unified framework for {Parameter-Efficient} transformers serving,” in *2022 USENIX Annual Technical Conference (USENIX ATC 22)*, 2022, pp. 489–504.
 - [80] Z. Zhou, X. Ning, K. Hong, T. Fu, J. Xu, S. Li, Y. Lou, L. Wang, Z. Yuan, X. Li *et al.*, “A survey on efficient inference for large language models,” *arXiv preprint arXiv:2404.14294*, 2024.